LiteBee Wing · LiteBee Bot (C10) · модуль позиционирования

# Как устроены дроны LiteBee

От метки на ковре до мотора:
разбор прошивки и протоколов

Книга собрана из разбора прошивок производителя (образ дрона Wing, ядро полёта C10, мобильные приложения), из файлов, снятых с самого дрона и с модуля позиционирования по USB, и из живых измерений на дроне 6 ноября 2026 года.

Каждое утверждение помечено: ПРОЧИТАНО (видно в файле или на устройстве), ВЫВОД (наше заключение) или НЕИЗВЕСТНО. Исходные тексты разбора лежат в папке `docs/firmware`, скрипты в `tools/firmware`.

# Оглавление

**Предисловие** 4

**1. Общая картина** 5
1.1. Кто что делает . . . 5
1.2. Две семьи прошивки . . . 5

**2. Дрон Wing изнутри** 7
2.1. Железо . . . 7
2.2. Флеш: восемь разделов . . . 7
2.3. Что запущено . . . 8
2.4. Доступ по USB . . . 8
2.5. Версии прошивки . . . 9

**3. Автопилот** 10
3.1. Слои . . . 10
3.2. Поток данных . . . 10
3.3. Режимы управления . . . 10
3.4. Миссия . . . 11
3.5. Отказы . . . 11
3.6. Журнал и параметры . . . 11

**4. Модуль позиционирования** 13
4.1. Железо . . . 13
4.2. Операционная система . . . 13
4.3. Программа . . . 13
4.4. Какие метки умеет искать модуль . . . 15
4.5. Три режима . . . 15
4.6. Датчик расстояния и светодиоды . . . 15
4.7. Что видит камера . . . 15
4.8. Подсветка: почему метка видна в темноте . . . 17
4.9. Что внутри образа флеша . . . 17
4.10. Как обновляется программа . . . 18

**5. Язык между модулем и дроном** 19
5.1. Кадр модуля . . . 19
5.2. Что делает дрон с кадром . . . 20
5.3. Версия заголовка: важная тонкость . . . 20
5.4. Команды дрона к модулю . . . 20
5.5. Что не выяснено . . . 21

**6. Ковёр: от номера метки до координат** 22
6.1. Номер метки равен номеру клетки . . . 22
6.2. Положение камеры . . . 22
6.3. Что говорит образ 1.2.8 . . . 23
6.4. Можно ли заменить AprilTag на ArUco . . . 23

**7. Приложения и протокол awlink** **24**
7.1. Кадр . . . 24
7.2. Группы . . . 25
7.3. Две формы команд управления . . . 25
7.4. Значения для Wing . . . 25
7.5. Телеметрия . . . 26
7.6. Наше приложение . . . 26

**8. Как мы это узнали** **27**
8.1. Хронология открытий . . . 27
8.2. Шаг 1. Файлы производителя . . . 27
8.3. Шаг 2. Приложения . . . 27
8.4. Шаг 3. Само устройство . . . 27
8.5. Подводные камни . . . 28
8.6. Инструменты в репозитории . . . 28

**9. Как делать свою прошивку** **29**
9.1. Три уровня . . . 29
9.2. Уровень 1: изменить программу модуля . . . 29
9.3. Уровень 2: своя камера . . . 30
9.4. Единицы . . . 30
9.5. Уровень 3: свой автопилот . . . 31
9.6. Чек-лист перед первым полётом . . . 31

**10. Тёмные пятна** **32**
10.1. Связь модуля с дроном . . . 32
10.2. Модуль . . . 32
10.3. Автопилот . . . 32
10.4. Протокол приложений . . . 33
10.5. Совсем не разобрано . . . 33
10.6. Что дальше даст больше всего . . . 33

**A. Файлы и документы** **34**

**B. Словарь** **35**

# Предисловие

Эта книга отвечает на вопрос, который сначала казался простым: *как дрон LiteBee Wing понимает, над какой клеткой ковра он находится*. Оказалось, что за ответом стоит целая цепочка. Камера на дроне, отдельный маленький компьютер под Linux, программа распознавания меток, кабель с собственным форматом кадра, автопилот со своей формулой и приложение, которое всё это показывает. Мы прошли эту цепочку от начала до конца и записали всё, что удалось подтвердить.

## Как читать книгу

Главы идут от общей картины к деталям. Если нужна только суть, читайте главы 1, 5 и 6. Если хотите писать свою прошивку или свою камеру, главы 9 и 10 скажут, что уже известно и где ещё тёмные пятна.

> **Пометки достоверности**
>
> Вся книга построена на трёх пометках, и мы стараемся не путать их.
>
> - ПРОЧИТАНО — факт виден прямо в файле, в дизассемблере, в памяти работающей программы или в ответе устройства.
> - ВЫВОД — наше заключение из прочитанного. Оно правдоподобно, но не проверено напрямую.
> - НЕИЗВЕСТНО — мы не знаем. Это честнее, чем гадать.

## Что у нас было

- Образы прошивки дрона Wing от производителя и пакеты обновления (Linux на Allwinner A33).
- Прошивка ядра полёта C10 (RISC-V) с отладочной информацией: имена функций и структур.
- Приложение LiteBee Go для компьютера и два мобильных приложения.
- **Сам дрон** и **сам модуль с камерой**, подключённые к компьютеру по USB.

> **Что нужно помнить**
>
> Дрон, на котором мы работали, использует прошивку марта 2023 года, а образ, который прислал производитель, новее (версия 1.2.8, 2025 год). Их автопилоты отличаются. Везде, где это важно, мы указываем, о какой версии идёт речь. Формула и формат кадра между ними совпали, а вот заголовок кадра отличается (глава 5).

> **Безопасность**
>
> Все эксперименты с дроном выполнялись **без пропеллеров** и только на чтение: ни во флеш дрона, ни во флеш модуля ничего не записывалось. Если вы повторяете что-то из книги, делайте так же.

Глава 1

# Общая картина

Дрон LiteBee Wing выглядит как одно устройство, но внутри и рядом с ним работают **три отдельных компьютера**. Каждый с собственной операционной системой и собственной программой.

## 1.1. Кто что делает

| Устройство | Чип и система | Задача |
|---|---|---|
| **Дрон Wing** | Allwinner A33, Linux 3.4-rt | Автопилот `pilot_f`: стабилизация, режимы, миссии; программа `flow` считает оптический поток; Wi-Fi и радиоканал пульта. |
| **Модуль позиционирования** | Allwinner V831, Linux 4.9 | Находит метку на ковре, меряет расстояние до земли, шлёт дрону готовый результат по UART. |
| **Приложение** | Windows, Android | Строит миссии, показывает телеметрию, обновляет прошивки. |

> **Главная находка**
>
> Опознание метки делает не дрон и не приложение, а **отдельный модуль**. В образе дрона нет ни одной таблицы меток. Автопилот получает от модуля уже готовое число: «видна метка номер такой-то, она смещена на столько-то, расстояние такое-то». ПРОЧИТАНО

## 1.2. Две семьи прошивки

Мы встретили сразу две платформы, построенные на одном и том же исходном коде автопилота (он называется *AWpilot* и пришёл от Allwinner).

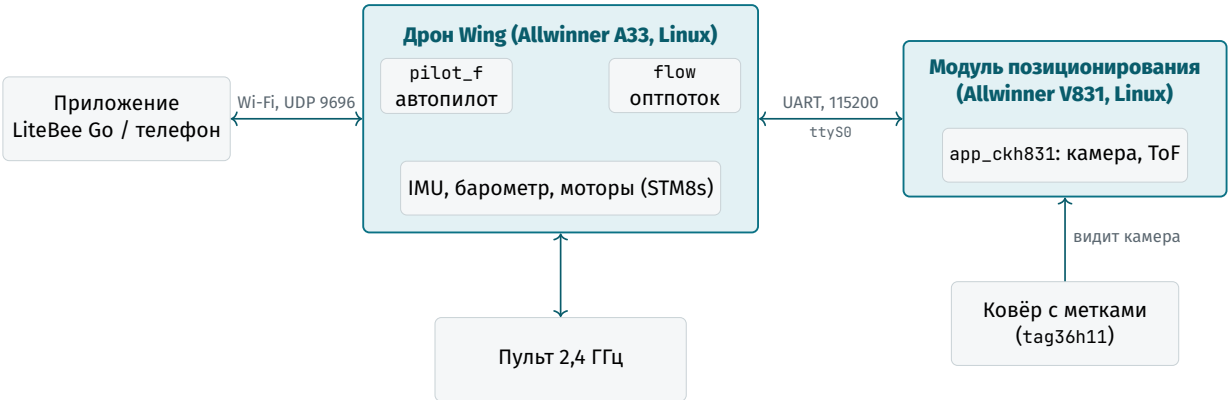



**Рис. 1.1.** Три компьютера и ковёр. Модуль смотрит на метки, автопилот летает, приложение рисует и командует.

| Семья | Чип полёта | Связь с приложением | Особенность |
|---|---|---|---|
| Wing (C06) | Allwinner A33, Linux | UDP 192.168.100.1:9696 | Автопилот `pilot`, бинарник без символов. |
| Bot (C10) | Allwinner V853S, ядро RISC-V (E907), Melis | TCP 192.168.5.1:17788 | Прошивка `melis30_1.5.elf` с символами и отладочной информацией. |

Прошивка C10 для нас как «чертёж»: в ней сохранены имена функций, структур и перечислений. Те же модули (и даже те же формулы) мы потом нашли в голом бинарнике Wing. ВЫВОД

> **Не путать**
>
> `melis30_1.5.elf` относится к C10, а не к Wing. Приложение LiteBee Go скачивает её как пакет `c10_drone_firmware`. Всё, что мы говорим про логику Wing «по прошивке ядра C10», основано на сходстве кода, а не на прямом чтении Wing. ПРОЧИТАНО

Глава 2

# Дрон Wing изнутри

Дрон Wing оказался не микроконтроллером с прошивкой, а настоящим маленьким компьютером с Linux. Это была самая неожиданная находка: на борту работает та же операционная система, что на роутере или одноплатнике, а полётная логика — обычная программа в ней.

## 2.1. Железо

| Узел | Что известно |
|---|---|
| Процессор | Allwinner семейства sun8i (класс A33), ядро Linux `3.4.39-rt143` (с патчем реального времени), имя узла `sun8i`. ПРОЧИТАНО |
| Память системы | Около 29 МБ оперативной памяти, из них свободно около 10 МБ при работающем автопилоте. ПРОЧИТАНО |
| Wi-Fi | Чип XR819 (драйверы `xradio`). В рабочем состоянии дрон выступает *клиентом* сети: работают `wpa_supplicant` и `udhcpc`. ПРОЧИТАНО |
| Датчики | Устройства `/dev/icm_imu` (инерциальный модуль), `/dev/dps280` (барометр), `/dev/STM8s` (драйвер моторов на микроконтроллере STM8). ПРОЧИТАНО |
| Камеры | `/dev/video0` и `/dev/video1`. Вторую, по конфигурации шины, читает программа оптического потока. ВЫВОД |
| UART | `ttyS0` (связь с модулем позиционирования), `ttyS2` (назначение не выяснено). ПРОЧИТАНО |

## 2.2. Флеш: восемь разделов

С живого дрона через ADB мы сняли побитовые копии всех разделов флеша (около 8 МБ в сумме).

| Раздел | Размер | Назначение |
|---|---:|---|
| `uboot` | 256 КБ | Загрузчик. |
| `boot` | 1320 КБ | Ядро Linux. |
| `rootfs` | 2416 КБ | Корневая файловая система (сжатая, только для чтения). |
| `env` | 128 КБ | Переменные загрузчика. |
| `Netprivate` | 128 КБ | Данные пользователя: параметры, Wi-Fi, файл `config.ini`. |
| `recoveryk` | 1320 КБ | Копия ядра для восстановления. |
| `recoveryf` | 2416 КБ | Копия корневой системы для восстановления. |
| `recoverye` | 128 КБ | Копия переменных загрузчика. |

Корень смонтирован как `squashfs` только для чтения. Поэтому «прошивка» дрона — это образ, который можно заменить целиком, а изменяемое состояние живёт в небольшом разделе `/netPrivate`. ПРОЧИТАНО

> **Почему это важно**
>
> Наличие парных разделов (`rootfs` и `recoveryf`) означает, что у дрона есть запасной вариант системы. Образ обновления от производителя нужен именно для этого механизма. Как работает переключение, мы не разбирали. НЕИЗВЕСТНО

## 2.3. Что запущено

С помощью списка процессов на живом дроне мы увидели, что работает по-настоящему:

| Процесс | Что делает |
|---|---|
| `/img_app/pilot/pilot_f` | Автопилот. Именно он (а не `pilot`) запущен на нашем дроне. ПРОЧИТАНО |
| `flow` | Считает оптический поток по картинке нижней камеры, результат отдаёт автопилоту через очередь сообщений. ПРОЧИТАНО |
| `wpa_supplicant`, `udhcpc` | Подключение к Wi-Fi как клиент. ПРОЧИТАНО |
| `adbd` | Отладочный сервис Android Debug Bridge: даёт оболочку по USB. ПРОЧИТАНО |
| `pmonitor`, `sunxi_poweroff_detection` | Служебные: наблюдение за питанием и процессами. ПРОЧИТАНО |
| `getty` на консоли 115200 | Последовательная консоль. ПРОЧИТАНО |

> **Что значит «pilot_f»**
>
> В каталоге `/img_app/pilot/` лежат два автопилота: `pilot` и `pilot_f`. Запущен второй. Буква, по всей видимости, означает вариант для формации (режим с несколькими дронами и клиентом Wi-Fi), но прямо это нигде не написано. ВЫВОД

## 2.4. Доступ по USB

Сервис `adbd` запускается при загрузке и показывается компьютеру как устройство `18D1:0002`. В образе есть и второй режим: диск на весь флеш (`18D1:0001`). Мы пользовались только ADB.

> **Питание**
>
> Дрон нельзя надёжно подключить только по USB: плата не поднимается. Нужен аккумулятор (при снятых пропеллерах). ADB при этом иногда отваливается через несколько минут. ПРОЧИТАНО

## 2.5. Версии прошивки

| Файл | На дроне | В образе 1.2.8 |
|---|---|---|
| `pilot` | 206 972 байта | 215 184 байта |
| `pilot_f` | 202 880 байт | 206 992 байта |
| Дата файлов | март 2023 | 2025 |

Прошивка на дроне старше образа от производителя. Отсюда следует практический вывод: чтобы понять, что *реально* работает на вашем дроне, нужно смотреть файлы с самого дрона, а не образ.
ПРОЧИТАНО

Глава 3

# Автопилот

Автопилот — это программа, которая читает датчики, решает, куда лететь, и управляет моторами. На Wing это `pilot_f`, на C10 — ядро Melis. Исходный код у них общий, поэтому прошивка C10 с отладочной информацией служит «картой» для Wing.

## 3.1. Слои

Прошивка C10 (287 единиц компиляции) устроена слоями, и мы нашли те же названия в бинарнике Wing.

## 3.2. Поток данных

1. Датчики (инерциальный модуль, барометр, оптический поток, дальномер, камера) отдают сырые значения.
2. Библиотеки оценивают ориентацию, высоту и положение.
3. Слой «куда лететь» выбирает цель.
4. Режим управления (один из нескольких) переводит цель в команду регуляторам положения, затем ориентации.
5. Драйвер моторов отдаёт мощности на четыре мотора.
6. Отдельно работает слой отказов и журнал.

## 3.3. Режимы управления

В таблице режимов прошивки C10 (по именам файлов исходников) мы нашли:

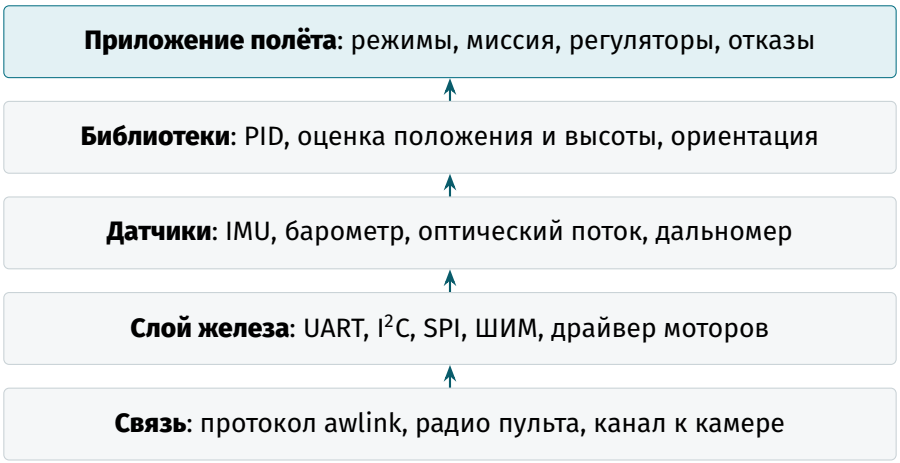



**Рис. 3.1.**. Слои автопилота. Данные идут снизу вверх, команды сверху вниз.

| Режим | Назначение |
|---|---|
| `stabilize`, `althold`, `poshold` | Ручной полёт, удержание высоты, удержание позиции. |
| `takeoff`, `land` | Взлёт и посадка (шаги миссии). |
| `mission` | Главный режим: выполняет шаги движения по порядку. |
| `circle`, `local360`, `jump` | Круг, оборот вокруг оси, «прыжок». |
| `track_lines` | Следование по линии (по камере). |
| `stop` | Остановка и выключение моторов. |

ПРОЧИТАНО Номера режимов, условия входа (например, «моторы взведены, высота известна») и переходы между ними разобраны в `docs/firmware/MISSION_AND_MODES.md`.

## 3.4. Миссия

Шоу дронов — это миссия: список шагов с метками времени. Шаг в прошивке занимает **18 байт**.

| Поле | Размер | Смысл |
|---|---|---|
| `count` | u16 | Номер шага. |
| `time` | u16 | Момент *окончания* шага, в десятых долях секунды от старта. |
| `type` | u8 | Команда: 0–14 (взлёт, перемещение, высота, круг, поворот и т.д.). |
| `data[6]` | 6 × int16 | Параметры шага (клетки, сантиметры, углы). |

ПРОЧИТАНО Это *ровно* тот кадр, который мы раньше восстановили по приложению: независимое подтверждение.

> **Время держит дрона, а не гонит его**
>
> Следующий шаг начинается, когда выполнены оба условия: цель текущего шага достигнута *и* до его конца осталось не больше 0,2 с. То есть время задаёт, когда дрон *отпускают* дальше, а не как быстро ему лететь. ПРОЧИТАНО (по ядру C10)

## 3.5. Отказы

По строкам автопилота видно, что он умеет реагировать на: потерю радиосвязи, потерю питания, низкий заряд, перегрев, превышение угла, столкновение, блокировку мотора, потерю положения. Для ситуации «положение потеряно» предусмотрены возврат домой и двухэтапная посадка. ПРОЧИТАНО

## 3.6. Журнал и параметры

Автопилот ведёт двоичный журнал с типами записей: `IMU`, `ATT`, `POS`, `VEL`, `BAT`, `FLOW`, `MSG`, `APRIL` (метка), `COLOR` и другие, всего около сорока. Каждая запись начинается с трёх байт заголовка.

Формат записи метки: время, три целых (`cx`, `cy`, `cz`), три числа курса и номер метки с флагом. ПРОЧИТАНО

На нашем дроне на земле в журнале были только записи `SYS` и `MSG`: записи положения пишутся, судя по всему, только в полёте. ВЫВОД

Параметры хранятся списком «имя–значение» в файлах `/netPrivate/param` и `param_calib`. У нашего дрона в `param` стоит одна строка: `MOTOR__GROUP_NUM 1.0000`. ПРОЧИТАНО

Глава 4

# Модуль позиционирования

Вторая половина системы — маленькая плата с камерой, которая смотрит вниз на ковёр. Мы сначала считали её «позиционным модулем» с неизвестной начинкой. Подключив её к компьютеру по USB, мы убедились, что внутри полноценный компьютер.

## 4.1. Железо

- Чип **Allwinner V831** (маркировка на кристалле): процессор Cortex-A7 с нейроускорителем. ПРОЧИТАНО
- Кнопки **BOOT** и **RST**, контакты 5V и GND, два разъёма с линиями TX и RX, USB. ПРОЧИТАНО
- Камера и датчик расстояния **ToF VI5300**, светодиоды **WS2812**. ПРОЧИТАНО

## 4.2. Операционная система

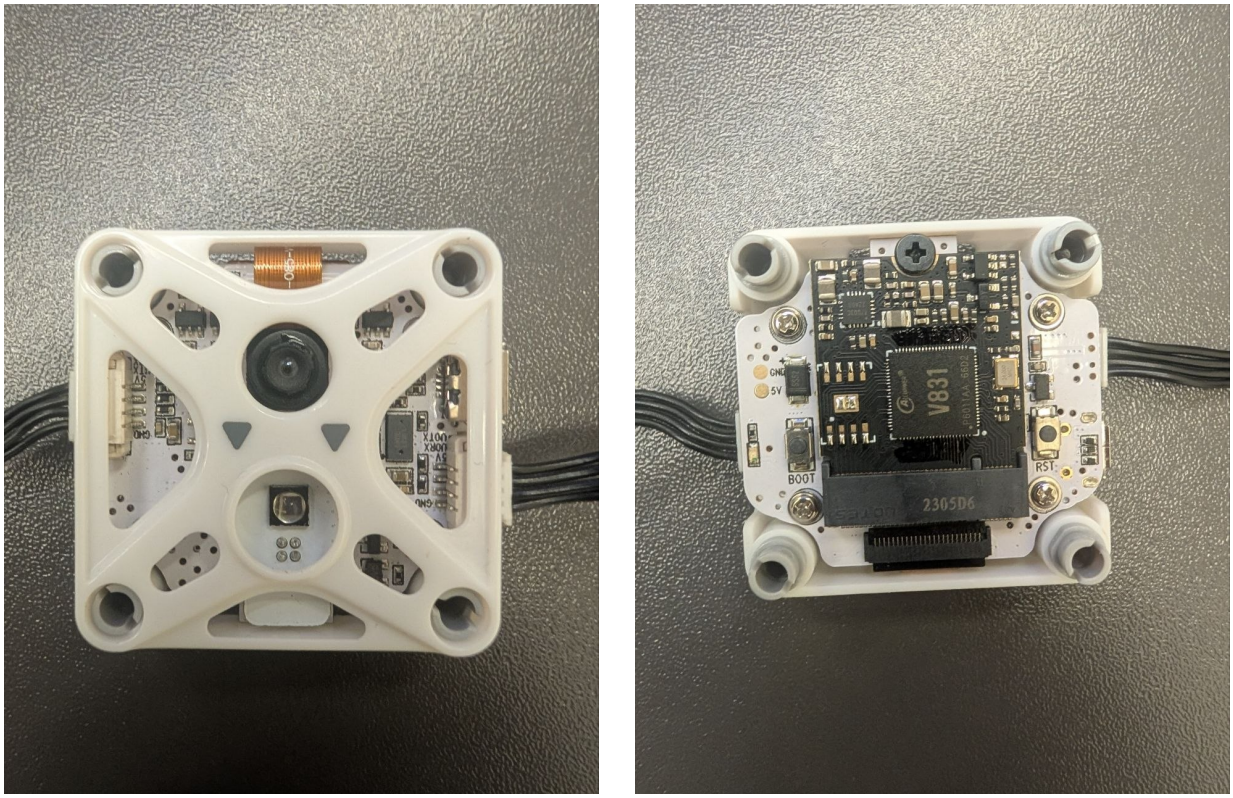
Подключённый к компьютеру модуль показывается как устройство `18D1:0002` (ADB) и как флешдиск на 50 МБ. Через ADB мы получили оболочку и прочитали систему.

| Параметр | Значение |
|---|---|
| Ядро | Linux 4.9.118, дистрибутив OpenWrt, собрано Sipeed, 32-бит ARM. ПРОЧИТАНО |
| Имя узла | `sipeed`. ПРОЧИТАНО |
| Библиотека C | musl, оболочка BusyBox, есть Python. ПРОЧИТАНО |
| Права доступа | `root` через ADB. ПРОЧИТАНО |
| Разделы | Корень 254 МБ, `/mnt/cfg` 69 МБ (программа и настройки), `/mnt/UDISK` 47 МБ (то, что видно как USB-диск). ПРОЧИТАНО |

## 4.3. Программа

Модулем управляет программа `app_ckh831` (всего 14 КБ) и большая библиотека `libmaix_ckh831.so` (1,5 МБ), в которой лежит вся работа. Динамические символы библиотеки не вычищены, поэтому мы увидели имена функций. Запускает программу цепочка скриптов: `S02app` → `keep_app.sh` (следит за процессом) → `app_ckh831`. ПРОЧИТАНО

> **Откуда взялся детектор**
>
> В библиотеке находится порт **imlib из OpenMV** — известной библиотеки компьютерного зрения для маленьких камер. Поиск меток AprilTag (`apriltag_detector_detect`), блобов и цвета — это функции imlib. Сверху добавлены протокол обмена с дроном, датчик расстоя-




(а) со стороны камеры (б) со стороны платы

**Рис. 4.1..** Модуль позиционирования Wing. Слева: объектив, светодиоды, датчик расстояния (круглое окно снизу), разъёмы с подписями 5V, TX, RX, GND. Справа: чип Allwinner V831, кнопки BOOT и RST, площадки 5V и GND, разъём камеры.

ния, светодиоды и графика. ПРОЧИТАНО

## 4.4. Какие метки умеет искать модуль

В библиотеку скомпилированы таблицы пяти семейств: `tag36h11`, `tag36h10`, `tag25h9`, `tag25h7`, `tag16h5`. Программа создаёт детектор вызовом `imlib_find_apriltags_new(0x10, ширина, высота)`. Первый аргумент — маска семейств, и в нумерации OpenMV значение `0x10` означает **только `tag36h11`**. Семейства ArUco в библиотеке нет. ПРОЧИТАНО

## 4.5. Три режима

Программа умеет работать в трёх режимах. Номер режима дрон выбирает специальной командой (глава 5).

| № | Режим | Функция в библиотеке |
|---|---|---|
| 0 | Поиск меток AprilTag | `get_imlib_find_apriltag_app` |
| 1 | Поиск блобов (цветных пятен, вероятно, для линии) | `get_imlib_find_blobs_app` |
| 2 | Поиск цвета | `get_imlib_find_color` |

ПРОЧИТАНО Порядок взят из таблицы перемещений самой библиотеки. Выбранный режим программа запоминает в JSON-конфигурации.

## 4.6. Датчик расстояния и светодиоды

Модуль меряет расстояние до пола датчиком ToF VI5300 по шине $I^2C$. Это расстояние уходит дрону в каждом кадре (раздел 5.1). Светодиоды WS2812 управляются командами дрона; у таких светодиодов порядок цветов *зелёный, красный, синий*, и ровно такой же порядок мы нашли в команде приложения `set_ledRGB`. ПРОЧИТАНО

## 4.7. Что видит камера

Чтобы увидеть то же, что видит модуль, мы остановили на нём программу детектора и сняли серию кадров стандартным Python-модулем `maix.camera`. Кадр имеет размер 320×240 (штатная программа работает с уменьшенным кадром 160×120). Модуль держали под углом около 45° над листом с меткой 13. ПРОЧИТАНО

Отсюда видно несколько вещей:

- В кадре сразу несколько меток (это три размера на одном листе). Штатная программа выбирает ту, что ближе к центру кадра.

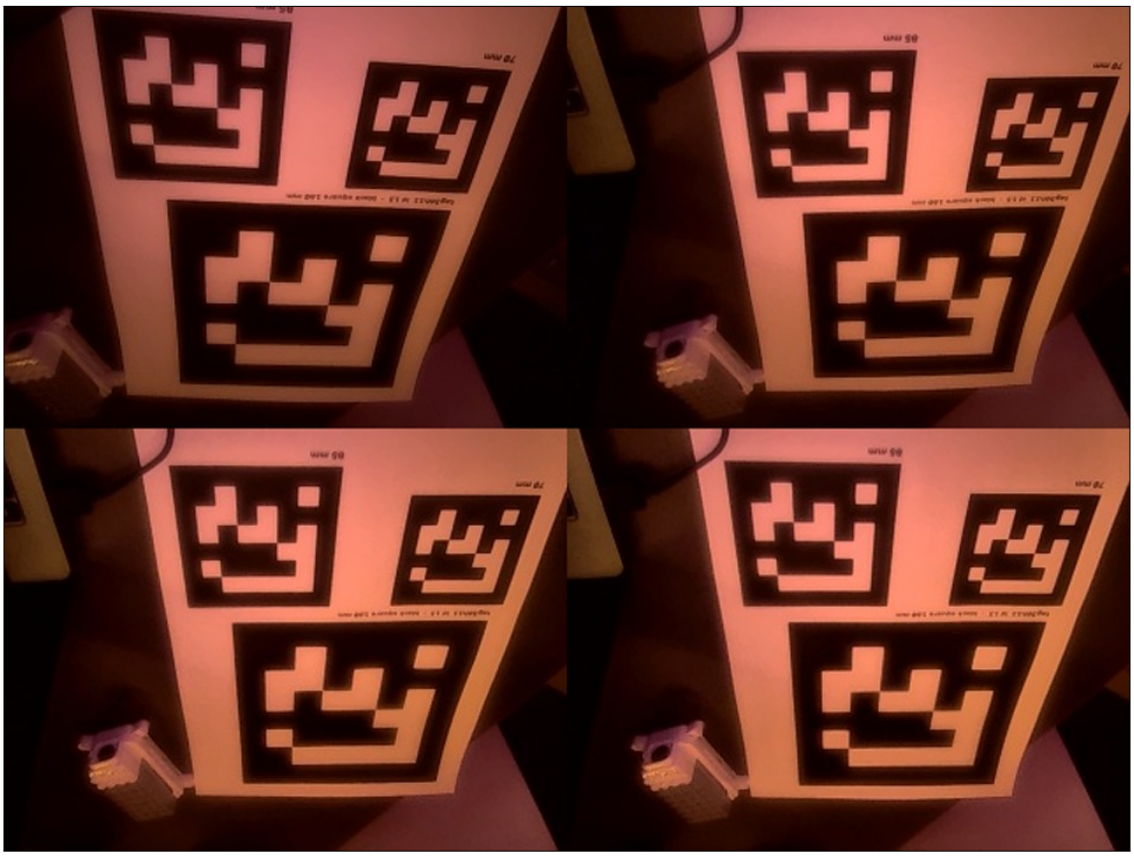


**Рис. 4.2..** Четыре кадра с камеры модуля (под углом около 45°). Лист в кадре повёрнут вверх ногами: на нём три одинаковые метки 13 размером 160, 85 и 70 мм. Розовый оттенок даёт собственная подсветка модуля.

- Метки хорошо читаются даже под углом. OpenCV на тех же кадрах нашёл номер 13 на всех десяти снимках (по две–три метки на кадре).
- Цвет с розовым оттенком: подсветка тёплая, камера без инфракрасного фильтра или с узкой полосой. ВЫВОД
- Лист был напечатан тремя размерами сразу, поэтому по этим кадрам нельзя сказать, какая именно метка использовалась в расчёте, и подтвердить размер около 100 мм.

## 4.8. Подсветка: почему метка видна в темноте

Владелец проверил: модуль опознаёт метку в полной темноте. Объяснение нашлось в самом модуле. В системе включены **четыре канала ШИМ** (период 10 мс, то есть 100 Гц) со скважностью **97,5 %**. Программа инициализирует ШИМ при старте и снова задаёт его при входе в режим меток, а команды дрона 4 и 6 тоже меняют ШИМ. ПРОЧИТАНО

Вывод (ВЫВОД): ШИМ управляет собственной светодиодной подсветкой модуля, которая светит почти на полную яркость. На фото платы по углам лицевой стороны виден ряд малых компонентов, похожих на ключи подсветки, но это предположение.

> **Для своей камеры**
>
> Освещение входит в систему. Камере, которая сама не светит, понадобится столько же света (или инфракрасная подсветка), иначе метки в темноте не распознаются. Яркостью модуля можно управлять командами дрона.

## 4.9. Что внутри образа флеша

Мы сняли побитовый образ всего флеша модуля (482 МБ) и разобрали его. Вывод оказался простым: **под программой производителя лежит готовый образ Sipeed MaixPy3**. ПРОЧИТАНО

| Раздел | Что внутри |
|---|---|
| `env`, `boot` | Переменные загрузчика и ядро (256 КБ и 4 МБ). |
| `cfg` (75 МБ) | Только программа производителя: каталог `firmware/maix_dist`. |
| `rootfs` (256 МБ) | Стандартный MaixPy3 0.5.4: Linux, Python 3.8, `numpy`, `pyserial`, `Pillow`, модуль `maix` (камера, ШИМ, GPIO, UART), модели нейросетей, библиотеки OpenCV 4.1 для C++. |
| `swap`, `UDISK` | Раздел подкачки и USB-диск (FAT). |

Из этого следует несколько практических вещей:

- В Python уже есть `find_apriltags` (тот же код imlib), а также поиск QR-кодов, линий, кругов и прямоугольников.
- Библиотека OpenCV `aruco` есть, но только для C++. Привязки `cv2` для Python нет.
- Серийный номер USB `20080411` и производитель `MaixPy3` задаются скриптом `S12usb`: поэтому он совпадает у модуля и у дрона (общий для платформы).




**Рис. 4.3.**. Обновление программы модуля через USB-диск.

- Скрипт запуска `S02app` запускает `/root/main.py` (файл на USB-диске модуля) вместо штатной программы. Это самый короткий путь к своей камере (глава 9). ВЫВОД

### Калибровка камеры

Параметры камеры лежат в самой программе как числа: кадр **160×120** пикселей, центр (80, 60), фокусное расстояние около **112,5 и 113,8** пикселя по осям. Из найденных меток выбирается та, что ближе к центру кадра. Выходные числа получаются так: положение метки умножается на $-100$, угол поворота в радианах умножается на 572, то есть получаются десятые доли градуса. ПРОЧИТАНО

По живым кадрам (расстояние метки 296–329 при ToF 287–311 мм) штатная программа ведёт себя так, как если бы метка имела сторону около 100 мм. Это вывод, а не прочитанное число. ВЫВОД

## 4.10. Как обновляется программа

Самое неожиданное в модуле — способ обновления. На USB-диске модуля лежат три файла: `maix_dist.sha256sum`, `V1.0.3` и `color_config.json`. Если рядом положить папку `maix_dist` с новой программой, то при следующем запуске скрипт `S02app` делает следующее. ПРОЧИТАНО

1. Видит файл `maix_dist/maix_dist.sha256sum`.
2. Копирует папку `maix_dist` во внутреннюю память (`/mnt/cfg/firmware`).
3. Удаляет её с диска.
4. Перезагружает модуль.

> **Сброс при ошибке**
>
> Если новая программа падает и запускается четыре раза подряд, сторожевой скрипт `keep_app.sh` вызывает `factory.sh`, то есть сбрасывает модуль. Это защита, но и риск: перед любыми изменениями нужно иметь полную копию. Мы её сняли (103 файла, около 30 МБ). ПРОЧИТАНО

Глава 5

# Язык между модулем и дроном

Модуль и автопилот соединены обычным последовательным кабелем: линии TX, RX, питание и земля. Скорость 115 200 бод, на стороне дрона это устройство `/dev/ttyS0`. Поверх кабеля идёт собственный простой протокол без названия. Мы восстановили его с трёх сторон: по коду модуля, по коду автопилота и по живому обмену. ПРОЧИТАНО

## 5.1. Кадр модуля

Модуль отправляет дрону кадр длиной **22 байта**. Первый байт всегда `CD`, последний — контрольная сумма (сумма байтов 1–20 по модулю 256).

| Байты | Поле | Смысл |
|---|---|---|
| 0 | заголовок | Всегда `CD`. |
| 1 | тип | `02` — кадр о метке. |
| 2–3 | номер метки | 16 бит, старший байт первым. Метка 0 дала `00 00`, метка 13 дала `00 0d`. |
| 4–5, 6–7 | смещения $x$, $y$ | Знаковые 16 бит, насколько камера сдвинута от центра метки. |
| 8–9 | расстояние | До метки, знаковые 16 бит. |
| 10–11 | угол | Поворот относительно метки, в десятых долях градуса. |
| 12 | флаг | 1 — метка найдена, 0 — нет. |
| 13–14 | счётчик | 16 бит (в наших кадрах 0). |
| 15–17, 20 | резерв | Нули. |
| 18–19 | ToF | Расстояние до пола в миллиметрах, **младший байт первым**. |
| 21 | сумма | Сумма байтов 1–20 по модулю 256. |

ПРОЧИТАНО Кадр снят дважды: сначала в памяти работающей программы модуля, потом уже *внутри* автопилота на дроне. Данные совпали байт в байт.




**Рис. 5.1.**. Живой кадр модуля, метка 0 под камерой. Нулевые байты 13–17 и 20 зарезервированы.

**Проверка в цифрах**

Метка 0 под камерой, наблюдаемый кадр: расстояние 296, ToF 287 мм (почти одно число), угол 3571 (то есть 357,1°, почти ноль, модуль был повёрнут ровно), смещения $-4$ и 22 (камера почти над центром метки). Всё согласуется. ПРОЧИТАНО

## 5.2. Что делает дрон с кадром

В автопилоте работает конечный автомат приёма. Он принимает байты из `/dev/ttyS0`, узнаёт заголовок, накапливает данные и проверяет сумму. Правильные данные копируются в буфер, и тип 2 запускает разбор метки. В памяти работающего `pilot_f` мы прочитали этот буфер: в нём лежали те же 20 байт данных, что отправлял модуль. У драйвера UART счётчик принятых байт рос: «20 152, 20 262». ПРОЧИТАНО

## 5.3. Версия заголовка: важная тонкость

Приёмник в старой прошивке дрона (март 2023) и приёмник в образе 1.2.8 (2025) *различаются*.

| Прошивка автопилота | Заголовок кадра от модуля |
|---|---|
| `pilot_f`, март 2023 (на нашем дроне) | Один байт: `CC` (14 байт данных) или `CD` (20 байт данных). |
| `pilot`, образ 1.2.8 (2025) | Два байта: `CC CC`, `CD CD` или `CE CE`. |

Модуль с программой, которая шлёт один байт `CD`, дружит со старой версией автопилота. Для образа 1.2.8 нужна программа модуля с двумя байтами заголовка. ПРОЧИТАНО (оба приёмника прочитаны); живая проверка сделана только со старой версией.

**Если делаете свою камеру**

Формат кадра нужно подбирать под *версию автопилота на дроне*. Сначала узнайте её (глава 8), потом пишите кадр.

## 5.4. Команды дрона к модулю

Дрон тоже шлёт модулю кадры. Они начинаются с двух байт `AD FA`, затем идёт команда, данные и сумма по данным. Разбор сделан по коду модуля (`ckh831_protocol_recv`).

| Команда | Что делает модуль |
|---|---|
| 1 | Выбирает режим (0 — метки, 1 — блобы, 2 — цвет). |
| 2 | Включает выход GPIO (линии `0xE2`, `0xE3`). |
| 3 | Задаёт цвет светодиодов WS2812. |
| 4, 6 | Устанавливает ШИМ на каналах 1 и 2 (во второй команде значение 16-битное). |
| 5 | Настраивает цвет для режима «цвет», запись в JSON. |
| 9 | Отвечает двумя байтами `F9 66` (проверка связи). |
| 10 | Делает снимок кадра. |
| `F1` | Устанавливает или сбрасывает внутренний флаг. |
| `FA` | Пересылает данные на второй порт (`ttyS1`). |

ВЫВОД Это разбор по коду. Какие из команд дрон шлёт на деле, мы не снимали: за пять минут на дроне в сторону модуля ушло 360 байт.

## 5.5. Что не выяснено

Другие типы кадров (1, 4, 5, `F9`) и кадр `CE CE` в автопилоте; смысл счётчика в байтах 13–14; зачем модуль пересылает данные на `ttyS1`. НЕИЗВЕСТНО

Глава 6

# Ковёр: от номера метки до координат

Ковёр — это сетка из меток `tag36h11`, и каждая метка несёт своё число. Автопилот превращает его в положение в комнате по очень простому правилу.

## 6.1. Номер метки равен номеру клетки

Клеток 12 в ряду. Номер метки совпадает с номером клетки (0–143), и по нему считаются строка и столбец:

$$\text{строка} = \lfloor id/12 \rfloor, \qquad \text{столбец} = id \bmod 12.$$

Шаг сетки **40 см**. Это подтверждено в бинарнике автопилота (функции деления на 12) и совпадает с нумерацией клеток в приложении. ПРОЧИТАНО

## 6.2. Положение камеры

Кроме номера, модуль присылает смещение камеры от центра метки. Автопилот складывает положение клетки и это смещение:

| Ось | Формула (метры) |
|---|---|
| Первая (по строке) | $\big(y \cdot 0{,}01 \cdot 8{,}333 + \text{строка} \cdot 40\big) \cdot 0{,}01$ |
| Вторая (по столбцу) | $\big(x \cdot 0{,}01 \cdot 8{,}333 + \text{столбец} \cdot 40\big) \cdot 0{,}01$ |
| Высота | расстояние $\cdot\, 10{,}2\cdot$ (малый коэффициент, около $10^{-4}$) |

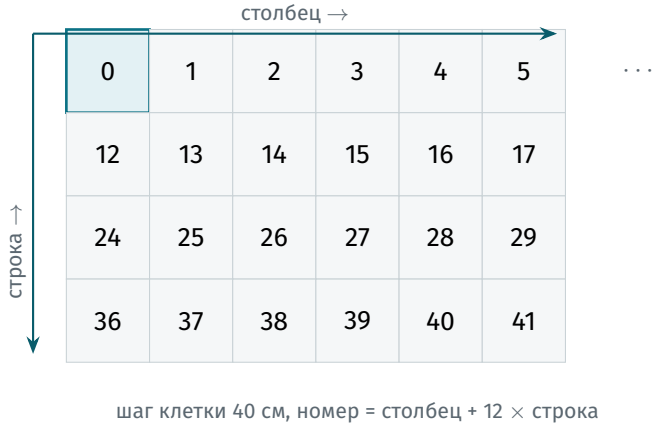



**Рис. 6.1..** Фрагмент ковра. Метка 0 — левый верхний угол; именно её мы клали под камеру.

ПРОЧИТАНО Константы 40, 8,333 и 10,2 лежат в сегменте данных автопилота по адресу `0x51200`. Угол приводится к диапазону $\pm 180^{\circ}$.

> **Проверка на живом дроне**
>
> Метка 13 лежит в клетке (строка 1, столбец 1). Автопилот принял кадр с $x = 108$, $y = -80$ и расстоянием 450. По формуле получается $0{,}3333$ и $0{,}4900$ м по осям и около $0{,}46$ м по высоте. Сам автопилот хранит в памяти **0,3327; 0,4908; 0,4627** м: совпадение с точностью около миллиметра. Единица $x$ и $y$ — примерно 0,83 мм, расстояния — примерно 1,02 мм. ПРОЧИТАНО

## 6.3. Что говорит образ 1.2.8

В образе 1.2.8 формула шире: есть режим «малых» меток (для номеров больше 99, шаг 5 и масштаб 2,083), особые константы для номеров 17–20 и проверка расстояния. В старой прошивке дрона, как мы видели, такого кода нет. ПРОЧИТАНО Почему номера 17–20 особые, мы не знаем. НЕИЗВЕСТНО

## 6.4. Можно ли заменить AprilTag на ArUco

Прямой ответ: **автопилоту всё равно**. Он принимает только число и смещения, а какое семейство меток эти числа породило, ему безразлично. Но между метками и автопилотом стоит модуль.

| Что хотите | Как |
|---|---|
| Другое семейство AprilTag (например `tag16h5`) | Поменять одну константу в программе модуля (маска `0x10`). Таблицы остальных четырёх семейств уже в библиотеке. ВЫВОД |
| ArUco | Готового кода нет. В модуле есть C++-библиотека `libopencv_aruco`, а Python-привязки `cv2` нет: нужна программа на C++ (кросс-компилятор под ARM). Её задача — слать кадр из главы 5. ВЫВОД |
| Другая сетка (не 12 колонок) | Константа 12 зашита в автопилоте; её пришлось бы менять там. ВЫВОД |

> **Размер меток**
>
> Размер метки и калибровка камеры влияют на расстояние и смещение. Где в программе модуля лежит калибровка, мы пока не выяснили, поэтому «заменить метки» нельзя считать готовым рецептом. НЕИЗВЕСТНО

Глава 7

# Приложения и протокол awlink

Дрон разговаривает с приложениями по Wi-Fi на протоколе **awlink**. Это короткие двоичные кадры с проверкой целостности. Один и тот же протокол используют Wing, C10 и наши собственные программы.

## 7.1. Кадр

| Поле | Байт | Смысл |
|---|---|---|
| `FA` | 1 | Начало кадра. |
| `len` | 1 | Число байт данных (0–255). |
| `src` | 1 | Отправитель: у приложения 0, у дрона 1; дрон его не проверяет. |
| `group` | 1 | Группа: 0 система, 1 статус, 2 управление, 3 миссия, 4 параметры, 5 журнал, 6 файлы. |
| `sub` | 1 | Команда в группе. |
| данные | `len` | Параметры. |
| CRC | 2 | CRC-16/MCRF4XX (начальное значение `FFFF`, многочлен `8408`), младший байт первым. |

ПРОЧИТАНО CRC проверена: функция в автопилоте даёт те же значения, что наша реализация, на двух тысячах случайных строк.

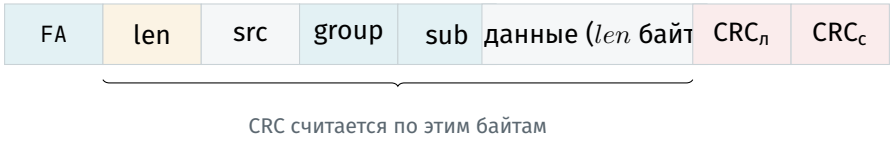



**Рис. 7.1.** Кадр awlink. Байт `FA` в сумму не входит; младший байт CRC идёт первым.

## 7.2. Группы

| № | Группа | Что внутри |
|---|---|---|
| 0 | система | Сердцебиение, версия, подтверждения. |
| 1 | статус | Телеметрия (только от дрона): базовая информация, датчики, калибровка, события. |
| 2 | управление | Стики, режимы, калибровка, высота, скорость, поворот. |
| 3 | миссия | Шаги миссии: приём и передача. |
| 4–6 | параметры, журнал, файлы | Чтение и запись параметров, выгрузка журнала, запись файла в дрон. |

## 7.3. Две формы команд управления

Здесь есть неожиданность, которую мы обнаружили, сравнив два мобильных приложения.

| Семья | Транспорт | Как кодируются движения |
|---|---|---|
| Wing (C06) | UDP, дрон `192.168.100.1:9696`, локальный порт 12345 | Режимы подгруппы 3 с кодами 20–27: перелёт в точку, направление, поворот, моторы, зуммер, светодиод. |
| C10 (Bot) | TCP, `192.168.5.1:17788` | Отдельные подгруппы: 12 (положение), 13 (скорость), 14 (поворот). |

ПРОЧИТАНО ВЫВОД Наша программа `litebee/live.py` построена по форме C10. Для Wing есть основание использовать форму подгруппы 3 или проверить на столе, какая из двух принимается. НЕИЗВЕСТНО

## 7.4. Значения для Wing

В блоках мобильного приложения значения кодируются так (прочитано в байткоде):

| Блок | Значения |
|---|---|
| Направление | 1 вперёд, 2 назад, 3 влево, 4 вправо, 5 вверх, 6 вниз, 7 стоп; скорость 1, 2 или 3. |
| Высота | Сантиметры (перелёт к высоте). |
| Поворот | 1 по часовой, 2 против; угол в градусах. |
| Светодиод | Лампа, затем зелёный, красный, синий (порядок GRB, как у WS2812). |
| Зуммер | Ноты: До 25, Ре 27, Ми 29, Фа 30, Соль 32, Ля 34, Си 36, До 37. |

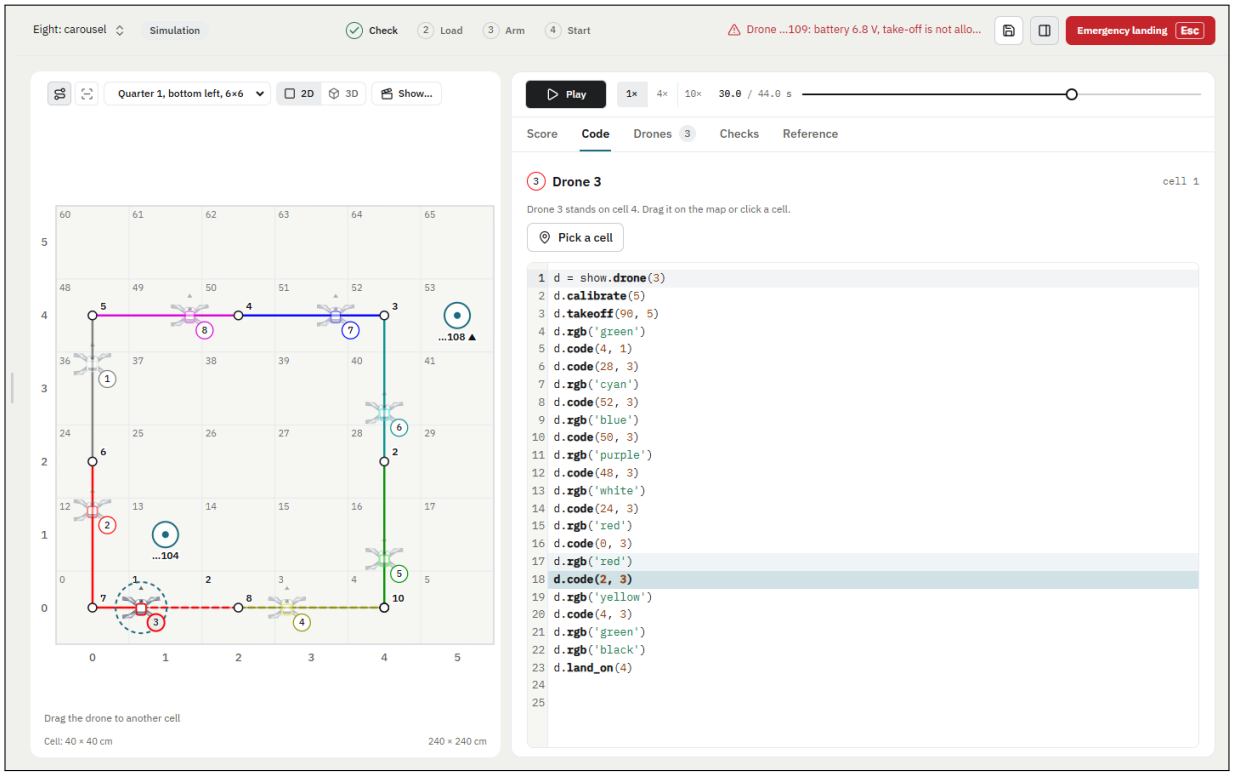



**Рис. 7.2.**. Litebee IDE: симуляция шоу для нескольких дронов.

## 7.5. Телеметрия

Кадр базовой информации (группа 1, подгруппа 0) несёт углы, скорость, положение, режим и заряд. Раскладка данных по смещениям от начала:

| Смещение | Тип | Поле |
|---|---|---|
| `0x00`–`0x08` | 3 × float | углы `attX`, `attY`, `attZ` |
| `0x0C`–`0x14` | 3 × float | скорость |
| `0x18`–`0x20` | 3 × float | положение |
| `0x24`–`0x2A` | 7 байт | статус, режим, ёмкость, напряжение, заряд, `headFree`, `armed` |
| `0x2C`, `0x30` | 2 × float | `locX`, `locY` |
| `0x34`, `0x35` | байты | `axisType` и версия |

ПРОЧИТАНО Приложение Wing читает режим из байта 42 кадра, а заряд из байта 43, что совпадает с этой раскладкой. Единицы полей (напряжение, ёмкость) мы не разбирали. НЕИЗВЕСТНО

## 7.6. Наше приложение

Параллельно с разбором прошивки мы писали собственную среду: Litebee IDE, где шоу пишут на Python и смотрят в симуляторе. Она использует тот же формат шага миссии, а на реальном дроне подключается по тому же протоколу.

Глава 8

# Как мы это узнали

Эта глава — рецепт. По ней можно повторить разбор на своём дроне и понять, где в нём доверять, а где перепроверять.

## 8.1. Хронология открытий

## 8.2. Шаг 1. Файлы производителя

Из образа дрона достаются корневые системы (`squashfs`), из них — программы. Для бинарников без символов используем дизассемблер (capstone). Главные приёмы:

- Строки: по ним сразу видны имена функций, форматы журнала, пути устройств.
- Ссылки на строки: функция, использующая строку «`use int_dev0`», — место, где включается этот канал.
- Ядро C10 с символами служит картой: находим там функцию по имени, затем её двойника в голом бинарнике.

## 8.3. Шаг 2. Приложения

Мобильные приложения (APK) содержат классы, построители кадров и собственные библиотеки. Для разбора использовались androguard (байткод) и дизассемблер для ARM64. Так мы получили команды Wing и раскладку телеметрии. ПРОЧИТАНО

## 8.4. Шаг 3. Само устройство

Файлы дают много, но не всё. Остальное дают сами устройства.

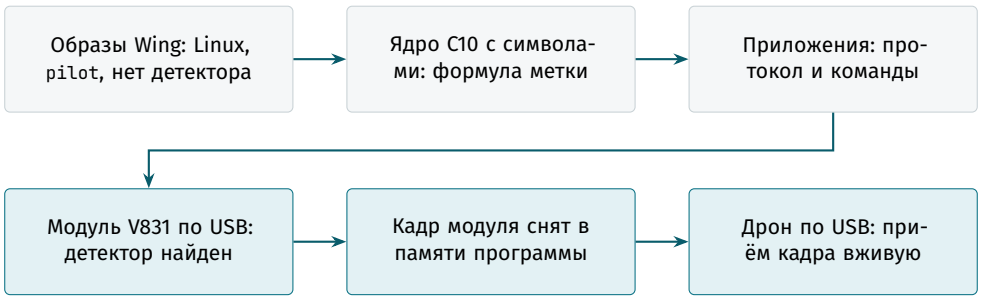



**Рис. 8.1.**. Порядок открытий. Закрашенные шаги дали ответы, которых не дали файлы.

1. **ADB.** Если устройство показывается компьютеру как `18D1:0002` и ADB его видит, есть оболочка. Команда `adb pull` копирует файлы. Подробнее в таблице ниже.
2. **Память процесса.** Читая `/proc/PID/mem` (только чтение), можно увидеть буфер, который автопилот заполнил из UART. Так мы проверили приём кадра.
3. **Счётчики.** Файл `/proc/tty/driver/` показывает число принятых и отправленных байтов каждого UART: сразу видно, говорит ли модуль с дроном.

| Что нужно узнать | Где смотреть |
|---|---|
| Какая версия автопилота стоит | Размер и дата `/img_app/pilot/pilot_f`. |
| Говорит ли модуль с дроном | Счётчик `rx` у `ttyS0`. |
| Что принял автопилот | Память процесса, буфер приёма. |
| Что шлёт модуль | Память программы модуля (последний кадр). |
| Какие режимы и команды понимает модуль | Таблица в библиотеке `libmaix_ckh831.so`. |

> **Правила работы**
>
> Только чтение. Без пропеллеров. Перед любыми изменениями — полные копии. Ни одной записи во флеш дрона и модуля. Питание дрона от аккумулятора (иначе он не загрузится).
>
> ПРОЧИТАНО

## 8.5. Подводные камни

- Серийный номер USB `20080411` одинаков у дрона и модуля: это номер по умолчанию от Allwinner. Различайте устройства по `uname` и списку процессов.
- Старый `adbd` на дроне не умеет `exec-out`: файлы берутся через `pull`, а разделы флеша копируются во временный файл в оперативной памяти.
- На дроне нет `od`, `tr`, `head`, `strings`: разбор байтов делается на компьютере.
- Линейный дизассемблер может пропускать куски кода. Мы один раз «не нашли» приёмник кадра, а он был на месте. Если поиск пустой, ищите другим способом.

## 8.6. Инструменты в репозитории

Скрипты лежат в `tools/firmware`: распаковка образов и приложений, дизассемблеры (ARM, ARM64, RISC-V), разбор отладочной информации, чтение классов Android-приложений, восстановление формулы метки, чтение буфера модуля. Все документы разбора лежат в `docs/firmware`.

ПРОЧИТАНО

Глава 9

# Как делать свою прошивку

Цель всей работы — понять, что можно менять. Мы теперь знаем достаточно, чтобы описать три уровня вмешательства от безопасного к рискованному.

## 9.1. Три уровня

## 9.2. Уровень 1: изменить программу модуля

Самый короткий путь к другому семейству меток. Что известно: маска семейств лежит в функции `imlib_find_apriltag_app_load` как константа `0x10`; остальные четыре семейства скомпилированы. Обновление идёт через USB-диск модуля. ПРОЧИТАНО

1. Снять полную копию модуля (она у нас есть).
2. Изменить константу в копии `libmaix_ckh831.so`.
3. Положить папку `maix_dist` на USB-диск и перезапустить модуль.
4. Проверить на дроне без пропеллеров, что дрон видит номер метки.

> **Ограничения**
>
> Калибровка камеры найдена (кадр 160×120, фокус около 112–114 пикселей), но размер метки в программе явно не задан: по живым кадрам он близок к 100 мм. Если программа падает четыре раза подряд, модуль сбросится.

### Проще: Python-скрипт вместо программы

Разбор образа флеша модуля показал более лёгкий путь. Корневая система модуля — стандартный образ Sipeed MaixPy3 (Python 3.8, `numpy`, `pyserial`, модуль `maix`). В нём уже есть

**Уровень 1. Данные вместо кода.** Менять рисунок ковра, другое семейство меток в программе модуля, параметры автопилота. Дрон трогать не нужно.

**Уровень 2. Своя камера вместо модуля.** Любая плата, которая шлёт автопилоту кадр из главы 5. Позволяет ArUco, QR и всё остальное.

**Уровень 3. Свой автопилот.** Замена `pilot_f`. Огромный проект: основой служат карта (`melis`) и логика (`pilot`).

**Рис. 9.1.** Лестница вмешательства: чем ниже, тем больше риск.

`find_apriltags` с теми же пятью семействами. Скрипт запуска `S02app` выполняет файл `/root/main.py` (то есть `main.py` на USB-диске модуля) *вместо* штатной программы. Значит, чтобы изменить семейство меток, не нужно править двоичный файл: достаточно положить на USB-диск скрипт, который ищет метки нужного семейства и шлёт дрону кадр из главы 5. ВЫВОД (по скрипту запуска; на живом модуле не проверялось). Чтобы вернуть штатную работу, удалите `main.py`.

## 9.3. Уровень 2: своя камера

Автопилот ничего не знает про меток: ему нужен только кадр. Свою камеру можно сделать из любой платы с UART, если она умеет:

- слать кадр из главы 5 с **правильным заголовком для версии автопилота** (один байт `CD` для версии марта 2023, два для 1.2.8);
- отвечать на команду проверки связи `9` (ответ `F9 66`);
- понимать команду выбора режима `1`, хотя бы режим 0;
- при желании управлять светодиодами и ШИМ.

Дальномер ToF (байты 18–19 кадра) нужно либо тоже поставить, либо слать нули; как автопилот использует это поле, мы не выяснили. НЕИЗВЕСТНО

> **Что даёт ArUco**
>
> Алгоритм ArUco из OpenCV находит метки другого семейства и даёт те же номера, смещения и угол. Если свой модуль посчитает их в единицах, которые ждёт автопилот (раздел 9.4), дрон полетит, как и по штатным меткам. На самом модуле библиотека `libopencv_aruco` есть, но только для C++: привязки `cv2` для Python в образе нет. ВЫВОД

## 9.4. Единицы

Для расчёта смещений в единицах автопилота (по формуле главы 5 и таблице ниже):

| Поле | Единица (приблизительно) |
|---|---|
| $x$, $y$ | 0,83 мм на единицу ($8{,}333 \cdot 10^{-4}$ м). |
| Расстояние | 1,02 мм на единицу ($10{,}2 \cdot 10^{-4}$ м). |
| Угол | 0,1°. |
| Номер | Целое от 0 до 287. |

ВЫВОД Единицы выведены из формулы автопилота и подтверждены одним живым кадром; для точности нужна серия замеров на разных высотах.

## 9.5. Уровень 3: свой автопилот

Полная замена автопилота — отдельный большой проект. Что у нас есть для него: формат всех кадров, структуры данных, режимы и их порядок, раскладка миссии, константы формулы метки, карта прошивки C10. Чего нет: исходного кода регуляторов, драйверов датчиков, закрытого алгоритма оптического потока, загрузчика и схем платы. НЕИЗВЕСТНО

## 9.6. Чек-лист перед первым полётом

1. Полная копия флеша дрона и программы модуля.
2. Проверка без пропеллеров: приём кадра (счётчик `rx`, буфер автопилота).
3. Проверка на привязи, на малой высоте.
4. Только потом свободный полёт и только на запасном дроне.

> **Безопасность**
>
> Ошибка в данных камеры напрямую влияет на управление полётом: дрон поверит неверному положению. Любое изменение в цепочке метка → кадр → автопилот проверяйте на столе, без пропеллеров.

Глава 10

# Тёмные пятна

Честный разбор состоит не только из ответов, но и из списка вопросов. Здесь собрано всё, что мы *не* знаем, и причины.

## 10.1. Связь модуля с дроном

- **Номер метки** (закрыто). С меткой 0 в байтах 2–3 стоял 0, с меткой 13 — `00 0d`, то есть 13. Привязка проверена на ненулевой метке. Не проверены только метки больше 255, где старший байт ненулевой (например, 143 помещается в один байт, а 287 нет). ПРОЧИТАНО
- **Другие типы кадров.** Типы 1, 4, 5, `F9` и кадр `CE CE` в автопилоте. НЕИЗВЕСТНО
- **Какие команды дрон шлёт модулю на деле** и с какой частотой. В программе модуля набор команд разобран, живой обмен не снят. НЕИЗВЕСТНО
- **Как автопилот использует расстояние ToF** (байты 18–19 кадра). НЕИЗВЕСТНО
- **Чем определяется выбор** между приёмником с одним или двумя байтами заголовка и между вариантами `int_dev0` и «ext device uart». Флаг найден, его значение по умолчанию нет. НЕИЗВЕСТНО

## 10.2. Модуль

- Содержимое образа флеша модуля (загрузчик, ядро, корневая система): образ снят целиком, но не разобран. НЕИЗВЕСТНО
- Калибровка камеры и размер метки: откуда берётся масштаб 8,333. НЕИЗВЕСТНО
- Зачем программа слушает `ttyS1` и пересылает байты. НЕИЗВЕСТНО
- Формат `color_config.json` (на модуле просто `null`); есть ли подпись у обновления (видна лишь контрольная сумма). НЕИЗВЕСТНО
- Совпадает ли эта программа с камерой C10. НЕИЗВЕСТНО

## 10.3. Автопилот

- Константы регуляторов, драйверы датчиков и исполнитель миссии известны по названиям и по ядру C10, но не по самому `pilot_f`. НЕИЗВЕСТНО
- Закрытый алгоритм оптического потока (`flow`). НЕИЗВЕСТНО
- Подпись образа дрона, возможность вернуть прежнюю прошивку. НЕИЗВЕСТНО
- Какая форма команд принимается вашим Wing: подгруппа 3 или подгруппы 12–14. НЕИЗВЕСТНО
- Как соотносятся USB-пульт и UDP-порт 9696. НЕИЗВЕСТНО
- Положение, посчитанное дроном: проверено вживую на метке 13, формула верна (глава 6). Не проверено на других высотах и при наклоне. ПРОЧИТАНО

## 10.4. Протокол приложений

- Единицы телеметрии (напряжение, ёмкость, углы). НЕИЗВЕСТНО
- «Версия» в байте 57 кадра по трассировке соответствует другому полю. НЕИЗВЕСТНО
- Порядок слотов моторов в команде `getEngine` (мотор 1, вероятно, в последнем слоте) и смысл `getAexsSelect`. НЕИЗВЕСТНО

## 10.5. Совсем не разобрано

Прошивка пульта (GD32E103), Linux-ядро C10 (V853S) и его камера, схемы плат, страницы документации производителя (не загрузились).

## 10.6. Что дальше даст больше всего

1. Снять живой обмен: что и как часто дрон шлёт модулю.
2. Разобрать снятый образ флеша модуля (загрузчик, ядро, настройки).
3. Проверить положение на нескольких высотах и при наклоне, а не на одной точке.
4. Разобрать приёмник автопилота `pilot` в режиме точки доступа: он отличается от `pilot_f`.

Приложение А

# Файлы и документы

Все материалы разбора лежат в репозитории и на диске. Исходные прошивки производителя в репозиторий не добавлялись.

## Документы в репозитории

| Файл | О чём |
|---|---|
| `docs/FIRMWARE_ARCHITECTURE.md` | Слои прошивки, потоки данных. |
| `docs/FIRMWARE_MARKERS.md` | Формула метки, ArUco. |
| `docs/FIRMWARE_UNKNOWNS.md` | Что известно и что нет. |
| `docs/firmware/AWLINK_TABLE.md` | Таблица команд awlink. |
| `docs/firmware/MISSION_AND_MODES.md` | Миссия и режимы. |
| `docs/firmware/WING_MOBILE_APP.md` | Приложение Wing, значения команд, телеметрия. |
| `docs/firmware/MODULE_UART_LINK.md` | UART между модулем и автопилотом. |
| `docs/firmware/POSITION_MODULE_V831.md` | Модуль: программа, кадр, команды. |
| `docs/firmware/WING_DRONE_USB.md` | Дрон по USB, живая проверка. |

## Папка с копиями прошивок (вне репозитория)

| Папка | Содержимое |
|---|---|
| `01_wing_drone_packages` | Три архива производителя. |
| `02_c10_melis_flight_core` | Прошивка ядра C10. |
| `03_position_module_V831` | Копия программы модуля, снята по ADB. |
| `04_mobile_apps` | Два APK. |
| `05_wing_rootfs_extracted` | Корневая система образа 1.2.8. |
| `06_wing_drone_flash_raw` | Восемь разделов флеша дрона. |
| `07_wing_drone_live_files` | Файлы с работающего дрона. |

Приложение B

# Словарь

**ADB**
: Android Debug Bridge. Отладочный канал по USB, даёт оболочку и копирование файлов. Запущен и на дроне, и на модуле.

**AprilTag**
: Семейство плоских меток-квадратов для компьютерного зрения. Наши ковры используют `tag36h11`.

**ArUco**
: Другое семейство меток из OpenCV. Штатный модуль его не распознаёт.

**awlink**
: Протокол кадров между дроном и приложениями (глава 7).

**CRC**
: Контрольная сумма для проверки целостности кадра.

**imlib**
: Библиотека компьютерного зрения из проекта OpenMV; на ней построен детектор модуля.

**Melis**
: Операционная система реального времени на ядре RISC-V дрона C10.

**MCRF4XX**
: Вариант CRC-16 с начальным значением `FFFF`, которым защищены кадры awlink.

**`pilot`, `pilot_f`**
: Программы-автопилоты на дроне Wing (`_f`: вариант, запущенный на нашем дроне).

**ToF**
: Time of Flight. Датчик расстояния по времени пролёта света; в модуле VI5300.

**UART**
: Простой последовательный интерфейс. На дроне соединяет автопилот с модулем (`ttyS0`, 115 200 бод).

**V831**
: Процессор Allwinner с нейроускорителем, на котором работает позиционный модуль.

**WS2812**
: Адресуемые светодиоды с порядком цветов зелёный, красный, синий.

**Формация**
: Режим нескольких дронов, в которых дрон подключается к сети как клиент.